

2057 Demographic Methods - Practical 1 (Population Composition and Demographic Rates)

2026-10-05

Synopsis

In this practical you will learn how to pull data from an online repository (GitHub) to create a population pyramid; to calculate excess population of each sex; and to answer a number of questions about the sex and age structure of that population (France, 1967), identifying characteristic shapes, population irregularities, and some of their underlying causes. This analysis is followed by an optional exercise related to demographic rates.

The heading of the R Script

The R Script starts by clearing all data if any and installing and loading required packages (in this example, lines are commented out using hashtags to prevent execution). This practical requires the package **ggplot2**, to draw plots; and **ggtext**, to extend ggplot2's text capabilities and improve the format of the figures.

```
rm(list = ls())
#install.packages("ggplot2")
#install.packages("ggtext")
library(ggplot2)
library(ggtext)
```

We can declare the URL from where the data will be downloaded, a GitHub repository in this case; and we can pull the data using the function **read.csv()**.

```
GitHub      = "https://raw.githubusercontent.com/Romero-Prieto/teaching/main/Demographic%20Methods/practical1/France1967.csv"
data        = read.csv(GitHub)
```

It is also good practice to use the first lines of a script to define some relevant characteristics of the population under study. For instance, we can create a **character** variable containing the name of the country and a **numeric** variable containing the corresponding year, as shown below. These variables can later be used for calculations, figure labels, and to construct the name of some outputs, e.g., data files, tables, and plots.

```
Country      = "France"
year         = 1967
```

Data inspection

One of the first steps is to learn about the structure of the data we are working with and their classification. Individual objects can be integers, numeric, logical, character, or date. More complex objects can be vectors, matrices, lists, or data frames. Knowing the class of an object is important because different classes have different properties, associated methods, and programming rules. We can use the function **class(object)** to learn the class of an object, as shown:

```
class(data)
```

```
## [1] "data.frame"
```

We can use the function **head(object, n)**, to print the first n rows of a given *object*; and the function **tail(object, n)**, to print the last n , as shown for $n = 5$:

```
head(data, 5)
```

```
##   Age      Male   Female
## 1    0 427988.5 409793.5
## 2    1 433498.5 415388.0
## 3    2 437974.0 419733.0
## 4    3 438601.5 421759.5
```

```
## 5    4 429450.0 413252.0
```

```
tail(data, 5)
```

```
##      Age  Male Female
## 97    96 627.0 2181.5
## 98    97 391.0 1431.5
## 99    98 212.5  811.5
## 100   99 115.0  482.0
## 101  100 110.5  620.0
```

As described, we are dealing with a **data frame** consisting of three columns: *Age*, *Male*, and *Female*, and 101 rows. This is an example of a **wide-form data**, in which each subpopulation is stored in a separate column, such as one column for each sex. More specific characteristics include the fact that age is reported in **single years (at last birthday)** and ranges from 0 to 100 years.

Demographic analysis and data preparation

All populations can be represented by age and sex, but not all population pyramids are created equal. Population pyramids provide indirect evidence of a long-term demographic history through the combined effects of fertility, mortality, and migration. Population deficits may reflect a past history of wars, epidemics, fertility declines, or sex-selective patterns of mortality and migration. Much of the demographic analysis that can be derived from population pyramids is based on comparisons of the same subgroup across different ages or of different subgroups at the same age. Hence, one particularly relevant assessment is the measurement of excess population by sex.

We can define the excess of male population as the difference between the maximum population across sexes and the female population, and the excess of female population as the difference between the maximum population across sexes and the male population, as shown:

```
data[, "Excess_M"] = pmax(data[, "Male"], data[, "Female"]) - data[, "Female"]
data[, "Excess_F"] = pmax(data[, "Male"], data[, "Female"]) - data[, "Male"]
```

Note that the function `pmax(vector_1, vector_2, ..., vector_n)` was used to calculate the maximum population across sexes for each age. We have used `data[, "Male"]` to extract the “Male” column from the data frame, while `data[, "Excess_M"] = ...` was indicated to create a new column named “Excess_M”. Keep in mind that the same results would be found using the alternative `$` notation, to the form `data$Male`.

The key to plotting population pyramids is to use horizontal bars and negative values to represent the age distribution of the male population (on the left side of the graph). Similarly, we can swap the sign of the excess female population which is also represented on the left side of the pyramid.

```
data[, "Male"]      = -data[, "Male"]
data[, "Excess_F"]  = -data[, "Excess_F"]
```

Plotting the population pyramid

To plot a population pyramid, we first need to define several parameters, including the age tick marks, the corresponding cohort labels, and the population numbers. To generate these sequences, we can use the function `seq(from, to, by)`. For instance, in the case of age, we consider the sequence of all multiples of 5 between 0 and 100, inclusive. Note that we identify the maximum age using the value in the last row of the Age column, i.e., `data[nrow(data), "Age"]`. The function `nrow(object)`, returns the total number of rows in an object. For the cohort, we simply take the difference between the year of the population and the age ticks; and for the population we define a set of arbitrary values that are appropriate for the population of France in 1967. Note that the sequence begins with negative values.

```
age_ticks      = seq(data[1, "Age"], data[nrow(data), "Age"], by = 5)
cohort_ticks   = year - age_ticks
population_ticks = seq(-500, 500, 100)
```

In addition to the key parameters, we can define the colours to be used—for example, one colour for each sex, and another to represent excess population. R uses the RGB model to represent colours by specifying the light intensities of *red*, *green*, and *blue* using values between 0 and 1. A fourth value specifies the level of transparency, **alpha**. We can therefore define a colour for each population in the data frame as follows:

```
alpha          = 0.40
fill_colours   = c("Excess_F" = rgb(1.00, 0.00, 0.00, alpha),
                  "Excess_M" = rgb(1.00, 0.00, 0.00, alpha),
```

```
"Male"      = rgb(0.45, 0.65, 0.20, alpha),
"Female"    = rgb(0.00, 0.55, 0.65, alpha))
```

Although wide-form data are more convenient for tabular representations, long-form data are generally more suitable for plotting and analysis because **all populations are stored in a single variable—with another variable identifying the population group, e.g., male or female**. Long-form data allow `ggplot()` to plot multiple populations using a single layer, the same aesthetics, and the same command. As a result, the code is simpler, more scalable, and easier to maintain. We generate a new data frame, `long_data`, using the `reshape()` function and specifying several arguments, including the direction of the transformation (`long`), the grouping variables, the name of the new value variable (`Population`), and the names of the groups (`Sex`). Once `long` has been created, use the function `View(long)` to inspect the structure of the data frame.

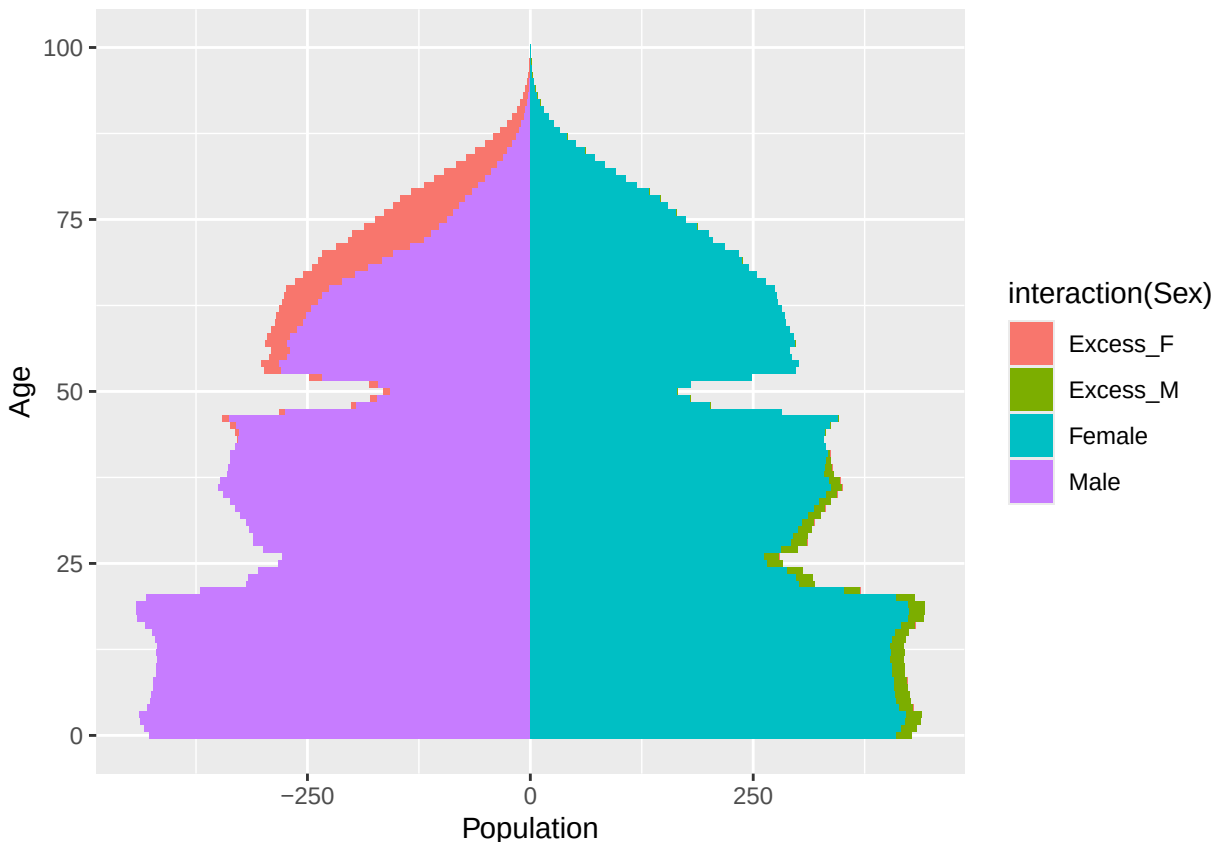
```
long = reshape(data, direction = "long",
               varying = c("Excess_F", "Excess_M", "Male", "Female"),
               v.names = "Population",
               timevar = "Sex",
               times = c("Excess_F", "Excess_M", "Male", "Female"))
```

To improve the readability of the figure, population counts are divided by 1,000 and expressed in thousands of persons.

```
long[, "Population"] = long[, "Population"]/1000
```

`ggplot2` is a layered graphics system in which plots are built by adding graphical layers to a common framework. Each layer represents a specific component of the figure, such as bars, points, lines, labels, or annotations. In particular, `ggplot()` defines the data and aesthetic mappings, `geom_col()` adds the bars, `coord_flip()` converts them into horizontal bars, `geom_text()` adds labels, and `labs()` adds titles and axis labels. It would be sufficient to use the following code to plot a population pyramid.

```
ggplot(long, aes(x = Age, y = Population, fill = interaction(Sex))) +
  geom_col(width = 1) +
  coord_flip()
```



However, we want the figure to follow a specific format, including dual axes to represent age and birth cohort, customised tick marks and colours, population values displayed as positive counts only, informative labels, and annotations indicating the units, data source, and the name of the population under study. Hence, we can define a `ggplot` object, `p`, and add layers to customise the axes using our predefined tick marks, modify the labels and legend, and include any annotations we consider

necessary.

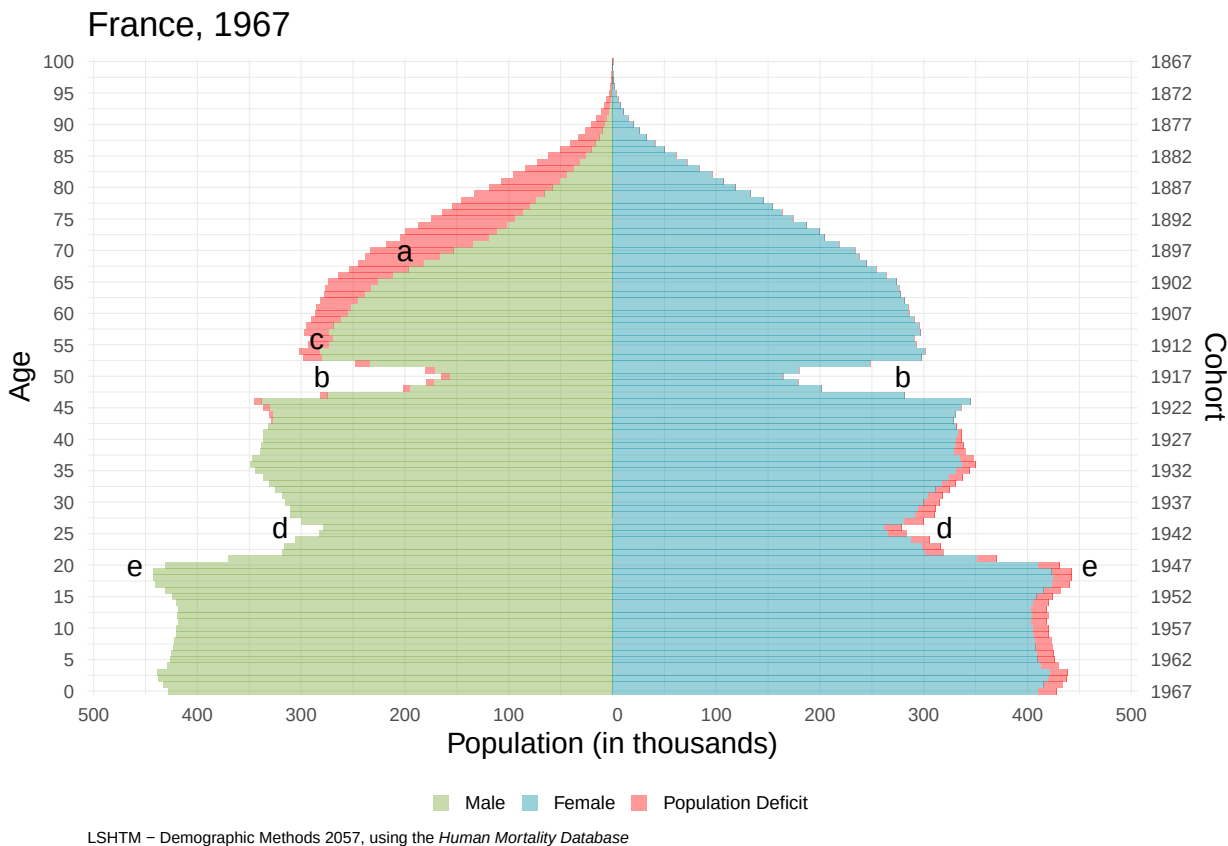
```
p = ggplot(long, aes(x = Age, y = Population, fill = interaction(Sex))) +
  geom_col(width = 1) +
  coord_flip() +
  scale_y_continuous(
    breaks = population_ticks,
    labels = function(x) format(abs(x), big.mark = ",", scientific = FALSE)) +
  scale_x_continuous(
    breaks = age_ticks,
    sec.axis = sec_axis(~ year - ., name = "Cohort", breaks = cohort_ticks),
    expand = c(0, 0)) +
  scale_fill_manual(
    values = fill_colours,
    breaks = c("Male", "Female", "Excess_M"),
    labels = c("Male", "Female", "Population Deficit")) +
  labs(
    title = paste0(Country, ", ", year),
    x = "Age",
    y = "Population (in thousands)",
    fill = "",
    caption = "LSHTM - Demographic Methods 2057, using the <i>Human Mortality Database<i>") +
  theme_minimal() +
  theme(
    axis.text.y = element_text(size = 7),
    axis.text.x = element_text(size = 7),
    legend.text = element_text(size = 7),
    legend.key.size = unit(0.25, "cm"),
    legend.position = "bottom",
    legend.margin = margin(0, 0, 0, 0),
    plot.caption = element_markdown(hjust = 0, size = 6),
    panel.grid.major = element_line(linewidth = 0.20),
    panel.grid.minor = element_line(linewidth = 0.10))
```

Note that we have created a ggplot object containing the data and the mappings between variables and visual elements, but the figure is not drawn immediately. Instead, **p** acts as a template to which additional layers, scales, labels, and themes can be added before the final plot is rendered. For example, we can add annotations highlighting the population irregularities of France in 1967, as shown below:

```
p = p +
  annotate("text", x = 70, y = -200, label = "a", size = 4) +
  annotate("text", x = 50, y = -280, label = "b", size = 4) +
  annotate("text", x = 50, y = 280, label = "b", size = 4) +
  annotate("text", x = 56, y = -285, label = "c", size = 4) +
  annotate("text", x = 26, y = -320, label = "d", size = 4) +
  annotate("text", x = 26, y = 320, label = "d", size = 4) +
  annotate("text", x = 20, y = -460, label = "e", size = 4) +
  annotate("text", x = 20, y = 460, label = "e", size = 4)
```

Finally, the figure can be displayed using **plot(p)**. By default, it will appear in the Plots tab of RStudio, located in the lower-right panel of the screen.

```
plot(p)
```



Exercise 1: Population Composition (examples)

Describe the most plausible causes of the population irregularities observed in France in 1967, as indicated by labels a to e, and answer questions f to g.

a. The deficit in the male population 70+ in 1967.

Military losses in WWI

b. The deficit in male and female births around 1916, resulting in a dent in the population pyramid at approximately age 51 in 1967.

Fertility postponement during WWI and the flu epidemic of 1918.

c. The deficit in the male population aged 45-69 in 1967.

Military losses in WWII.

d. The deficit in male and female births around 1942, resulting in a dent in the population pyramid at approximately age 25 in 1967.

Fertility postponement during WWII.

e. The increase in the number of births around 1947, resulting in a population hump at approximately age 20 in 1967.

Baby boom after WWII, thus the war could cause a tempo distortion.

f. True or false: “The male-to-female sex ratio of France in 1967 was approximately 1.05.” Explain your answer.

False: the sex ratio at birth is usually around 1.05, but male mortality rates are higher for men than for women and that usually results in sex ratios in the population as a whole that are below unity. The sex structure of late 20th century France was also heavily affected by the war which has had many more male than female casualties.

g. What could be the cause of the excess male population at all ages below 40?

In a closed population, an excess male population is expected to disappear by early adulthood as a result of higher male mortality. Therefore, an excess male population sustained at all ages below 40 may be indicative of sex-selective migration at working ages, such as relatively higher male in-migration or relatively higher female out-migration.

Questions requiring some R calculations (examples)

1. Calculate the sex ratio for the population as a whole.

```
data = abs(data[, c("Age", "Male", "Female")])
sum(data[, "Male"])/sum(data[, "Female"])
```

```
## [1] 0.9494593
```

2. Calculate the sex ratio for the under-5 population.

```
sum(data[data[, "Age"] < 5, "Male"])/sum(data[data[, "Age"] < 5, "Female"])
```

```
## [1] 1.04211
```

3. Calculate the sex ratio at age 0 and explain why this value is close to, but not equal to, the sex ratio at birth.

```
sum(data[data[, "Age"] == 0, "Male"])/sum(data[data[, "Age"] == 0, "Female"])
```

```
## [1] 1.0444
```

While population at age 0 is a stock, the number births constitute a flow. Therefore, the age-0 population depends on the number of births and the age distribution of deaths during the first year of life. Boys could experience higher mortality than girls during the first year of life. Migration may also play a small role.

4. Calculate the sex ratio below each age, from 0 to 100+

```
cumsum(data[, "Male"])/cumsum(data[, "Female"])
```

```
## [1] 1.0444004 1.0439970 1.0438155 1.0428329 1.0421104 1.0418498 1.0414931
## [8] 1.0410887 1.0406730 1.0402983 1.0400195 1.0397713 1.0395654 1.0393793
## [15] 1.0392530 1.0391607 1.0391216 1.0391948 1.0393947 1.0397381 1.0402438
## [22] 1.0407782 1.0413008 1.0418998 1.0425134 1.0430953 1.0436722 1.0442671
## [29] 1.0447788 1.0450990 1.0452665 1.0453036 1.0452618 1.0451708 1.0450554
## [36] 1.0449162 1.0447665 1.0445725 1.0442792 1.0438467 1.0432576 1.0425054
## [43] 1.0416364 1.0406703 1.0396087 1.0384440 1.0372399 1.0362241 1.0354164
## [50] 1.0346405 1.0338794 1.0330142 1.0317710 1.0302763 1.0287515 1.0272473
## [57] 1.0257249 1.0241053 1.0224309 1.0206881 1.0188869 1.0170220 1.0150713
## [64] 1.0130213 1.0108425 1.0085099 1.0060347 1.0033996 1.0005567 0.9974243
## [71] 0.9939674 0.9904258 0.9868811 0.9832770 0.9798395 0.9766312 0.9735908
## [78] 0.9706885 0.9679157 0.9653463 0.9630553 0.9609578 0.9590317 0.9573179
## [85] 0.9557945 0.9544743 0.9533803 0.9524759 0.9517351 0.9511379 0.9506669
## [92] 0.9503057 0.9500377 0.9498427 0.9496989 0.9496085 0.9495517 0.9495136
## [99] 0.9494916 0.9494781 0.9494593
```

Exercise 2: Demographic Rates (Optional)

The following rates were recorded for a fictive population: $CBR = 0.0145$; $CDR = 0.0078$; $CGR = 0.0085$. Compute the Crude Rate of Natural Increase ($CRNI$) and the Crude Rate of Net Migration ($CRNM$).

```
CBR          = 0.0145
CDR          = 0.0078
CGR          = 0.0085
CRNI         = CBR - CDR
sprintf("the CRNI is %.2f per 1,000", CRNI*1000)
```

```
## [1] "the CRNI is 6.70 per 1,000"
```

```
CRNM         = CGR - CRNI
sprintf("the CRNM is %.2f per 1,000", CRNM*1000)
```

```
## [1] "the CRNM is 1.80 per 1,000"
```

Note that we have used the function `sprintf("text + % + format + f + more text", value)`, where `%f` is a placeholder for a numeric value, to report the $CRNI$ and $CRNM$ using predefined text and formatted output.

Optional steps to save your results

You may want to save your outputs at a specific location on your computer, and the first step is to identify your default working directory, using the function `getwd()`, as shown below:

```
getwd()
```

```
## [1] "/Users/lshjr3/Documents/Demographic_Methods/Practical_1"
```

You can change the working directory using the function `setwd(path)`, providing a new path to the folder, for example:

```
setwd("~/Documents/Demographic_Methods/Practical_1")
getwd()
```

```
## [1] "/Users/lshjr3/Documents/Demographic_Methods/Practical_1"
```

Then, you can save the resulting population pyramid to that specified location using `ggsave()`, just providing specif width and height of the figure in inches.

```
ggsave("France_1967.png", plot = p, width = 5, height = 4)
```

Note that you have previously created variables to define the country and year of this population, and you could use them to construct the name of the output file. For example, you can use `paste0(string_1, ..., string_n)` to concatenate multiple characters and numbers without adding any spaces or separators between them, as shown:

```
paste0(Country, "_", year, ".png")
```

```
## [1] "France_1967.png"
```

```
ggsave(paste0(Country, "_", year, ".png"), plot = p, width = 5, height = 4)
```

Finally, you may want to follow the best practice of saving files in a dedicated folder rather than in the default working directory. This approach is often more convenient because data and output files can be organized in separate locations, while the working directory can be reserved for other purposes. To do so, you can define the path to a specific folder and use it when saving outputs. As shown below, this can once again be accomplished using the `paste0()` function:

```
pAth          = "~/Documents/Demographic_Methods/Practical_1/"
ggsave(paste0(pAth, Country, "_", year, ".png"), plot = p, width = 5, height = 4)
```